



UNIVERSITÉ TOULOUSE III

RAPPORT DE PROJET

Image, Signal, Simulations

Sujet 1: Demosaïcage d'image couleurs

Milhane Issimailla Sabiti
Nablaoui Wafaa
Boukaref Aya
Diallo Moussa

Contents

1	Introduction	3
2	Filtre de Bayer	4
2.1	Implémentation de l'opérateur F	4
2.2	Calcul de l'opérateur adjoint F^*	4
2.3	Illustrations des résultats	5
3	Problème de minimisation	6
3.1	Définition du problème	6
3.2	Étude de la Différentiabilité, Convexité et Coercivité du problème	6
3.3	Calcul du gradient de TV_ϵ	8
3.3.1	Éléments préliminaires de calcul	8
3.3.2	Calcul du gradient;	9
4	Algorithme de Restauration d'image et Résultats	11
4.1	Implémentation de la fonction <i>démosaïcage</i>	11
4.2	Convergence des algorithmes	12
4.3	Test avec différentes Images	13
5	Conclusion	15
6	Sources	16

1 Introduction

Le traitement d'image est un domaine en perpétuelle évolution, à l'intersection de l'informatique et des mathématiques appliquées. Cette discipline se consacre à l'étude, la transformation et l'analyse des images numériques afin d'en améliorer la qualité ou d'extraire des informations pertinentes. Aujourd'hui, le traitement d'image joue un rôle crucial tant dans notre vie quotidienne que dans le monde de la recherche scientifique, avec des applications allant de l'amélioration des imageries médicales à la retouche photographique.

Dans ce projet, nous explorerons le défi du démosaïçage d'images couleur, également connu sous le nom d'interpolation CFA (Color Filter Array). Ce procédé repose sur un algorithme de traitement d'images numériques qui vise à reconstruire une image couleur à partir d'échantillons de couleurs incomplets fournis par un capteur d'image équipé d'un motif de filtres colorés, tel que le filtre de Bayer. Grâce à cet algorithme, il est possible de recréer une image couleur à partir d'une image en niveaux de gris ayant les mêmes dimensions. Pour garantir une reconstruction de qualité, il est essentiel d'éviter l'apparition de fausses couleurs c'est-à-dire des variations brusques et artificielles d'intensité sur plusieurs pixels voisins. Par ailleurs, l'algorithme doit préserver au maximum la résolution de l'image et faciliter l'analyse pour réduire précisément le bruit.

Notre étude se concentrera sur le filtre de Bayer, la configuration CFA la plus couramment utilisée. Une mosaïque de filtres Bayer est un réseau de filtres de couleur (CFA) permettant d'organiser des filtres de couleur RVB (Rouge, Vert, Blue) sur une grille carrée de photodétecteurs. L'objectif est donc de reconstituer l'image couleur complète à partir de ces données partielles. Concrètement, cela implique de procéder à une interpolation des valeurs manquantes en s'appuyant sur les pixels voisins, en utilisant des méthodes mathématiques et algorithmiques pour inverser le filtre de Bayer.

2 Filtre de Bayer

Le filtre de Bayer, couramment utilisé dans les capteurs couleur, divise l'image en une matrice de pixels où chaque pixel est ne détecte qu'une seule couleur rouge, vert ou bleu selon sa position dans la grille.

2.1 Implémentation de l'opérateur F

Le filtre de Bayer est un opérateur linéaire F définie de $(\mathbb{R}^3)^{N^2} \longrightarrow \mathbb{R}^{N^2}$ tel que $v = Fu$.

avec $u = (u_{i,j}^r, u_{i,j}^g, u_{i,j}^b)$ et

$$v_{i,j} = \begin{cases} u_{i,j}^b, & \text{si } i[2] = 0 \text{ et } j[2] = 0, \\ u_{i,j}^r, & \text{si } i[2] = 1 \text{ et } j[2] = 1, \\ u_{i,j}^g, & \text{sinon.} \end{cases}$$

L'implémentation de cet opérateur en langage python, nous donne le code suivant:

```

1 # Implémentation de l'opérateur F
2 def F(img):
3     k, l = (img.shape[0], img.shape[1])
4     v = np.zeros((k, l))
5
6     # Parcours des lignes et des colonnes de l'image
7     for i in range(k):
8         for j in range(l):
9             # Extraction des couleurs
10            if i % 2 == 0 and j % 2 == 0:
11                v[i, j] = img[i, j, 2]
12            elif i % 2 == 1 and j % 2 == 1:
13                v[i, j] = img[i, j, 0]
14            else:
15                v[i, j] = img[i, j, 1]
16
17     return v

```

Listing 1: Implémentation de F

2.2 Calcul de l'opérateur adjoint F*

Soit $u \in (\mathbb{R}^3)^{N^2}$ et $v \in \mathbb{R}^{N^2}$. On note $\langle, \rangle$ le produit scalaire euclidien, on a alors:

$$\begin{aligned} \langle F(u), v \rangle &= \sum_{i,j=1}^N \begin{cases} u_{i,j}^b \cdot v_{i,j}, & \text{si } i[2] = 0 \text{ et } j[2] = 0, \\ u_{i,j}^r \cdot v_{i,j}, & \text{si } i[2] = 1 \text{ et } j[2] = 1, \\ u_{i,j}^g \cdot v_{i,j}, & \text{sinon.} \end{cases} = \sum_{i,j=1}^N \begin{cases} u_{i,j} \cdot (0, 0, v_{i,j}), & \text{si } i[2] = 0 \text{ et } j[2] = 0, \\ u_{i,j} \cdot (v_{i,j}, 0, 0), & \text{si } i[2] = 1 \text{ et } j[2] = 1, \\ u_{i,j} \cdot (0, v_{i,j}, 0), & \text{sinon.} \end{cases} \\ &= \langle u, F^*(v) \rangle \end{aligned}$$

L'opérateur adjoint de F est donc définie :

$$F^* : \mathbb{R}^{N^2} \longrightarrow (\mathbb{R}^3)^{N^2}$$

$$F^*(v)_{i,j} = \begin{cases} (0, 0, v_{i,j}), & \text{si } i[2] = 0 \text{ et } j[2] = 0, \\ (v_{i,j}, 0, 0), & \text{si } i[2] = 1 \text{ et } j[2] = 1, \\ (0, v_{i,j}, 0), & \text{sinon.} \end{cases}$$

Après avoir calculé l'adjoint de F et implémenter, on obtient le code suivant:

```

1 # Implémentation de l'adjoint de l'opérateur F
2 def F_adjoint(image):
3     k, l = np.shape(image)
4     v = np.zeros((k, l, 3))
5

```

```

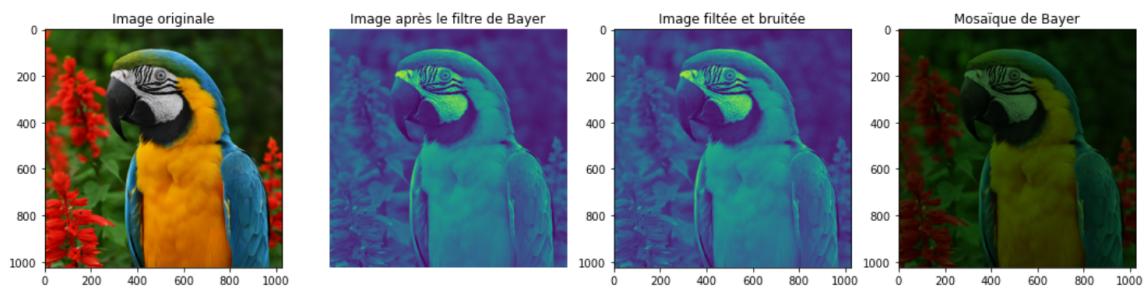
6  # Parcours des lignes et des colonnes de l'image
7  for i in range(k):
8      for j in range(1):
9          if i % 2 == 0 and j % 2 == 0:
10             v[i, j, 2] = image[i, j]
11          elif i % 2 == 1 and j % 2 == 1:
12             v[i, j, 0] = image[i, j]
13          else:
14             v[i, j, 1] = image[i, j]
15
16  return v

```

Listing 2: Implémentation de F_{adjoint}

2.3 Illustrations des résultats

Nous effectuons un test de nos algorithmes et nous obtenons les résultats suivants:



3 Problème de minimisation

3.1 Définition du problème

On considère un signal v résultant d'une transformation linéaire F appliquée à un signal original u , affecté par un terme de bruit additif b , tel que $v = Fu + b$.

Pour une image couleur $w \in (\mathbb{R}^3)^{N^2}$, on définit un terme de régularisation basé sur la variation totale, noté $TV_\varepsilon(w)$, comme :

$$TV_\varepsilon(w) = \sum_{i,j=0}^{N-1} \sqrt{(\partial_m w^r)_{i,j}^2 + (\partial_n w^r)_{i,j}^2 + (\partial_m w^g)_{i,j}^2 + (\partial_n w^g)_{i,j}^2 + (\partial_m w^b)_{i,j}^2 + (\partial_n w^b)_{i,j}^2} + \varepsilon,$$

où $\varepsilon > 0$ est un paramètre de lissage et ∂_m et ∂_n représentent les opérateurs de différences finies discrètes selon les directions m et n , appliqués aux composantes rouge (w^r), verte (w^g), et bleue (w^b) de l'image.

Le problème d'optimisation est alors formulé comme suit :

$$(P) \begin{cases} \min E(w) = TV_\varepsilon(w) + \lambda \|Fw - v\|^2 \\ s.c \quad w \in (\mathbb{R}^3)^{N^2} \end{cases}$$

L'existence et l'unicité d'un minimiseur sur l'espace $(\mathbb{R}^3)^{N^2}$ sont garanties par les propriétés de $E(w)$.

3.2 Étude de la Différentiabilité, Convexité et Coercivité du problème

Différentiabilité de $TV_\varepsilon(w)$ et Différentiabilité de $\lambda \|Fw - v\|^2$

Comme on sait que

$$\partial_m w^r, \partial_n w^r, \dots \text{ sont linéaires } \implies \text{ différentiables.}$$

$$(\partial_m w^r)_{i,j}^2 \text{ est une composée de fonctions différentiables } \implies \text{ différentiable.}$$

$$S(w) = (\partial_m w^r)_{i,j}^2 + (\partial_n w^r)_{i,j}^2 + (\partial_m w^g)_{i,j}^2 + (\partial_n w^g)_{i,j}^2 + (\partial_m w^b)_{i,j}^2 + (\partial_n w^b)_{i,j}^2 + \varepsilon$$

est une somme de fonctions différentiables $\implies S(w)$ est différentiable.

$$S(w) > 0 \text{ et } x \mapsto \sqrt{x} \text{ est différentiable sur } \mathbb{R}_+^*.$$

$$TV_\varepsilon(w) = \sum_{i,j=0}^{N-1} \sqrt{S(w)}$$

est une somme de composées de fonctions différentiables $\implies TV_\varepsilon(w)$ est différentiable.

$$Fw \text{ est linéaire } \implies \text{ différentiable.}$$

$$Fw - v \text{ est une translation } \implies \text{ différentiable.}$$

$$\|Fw - v\|^2 = (Fw - v) \cdot (Fw - v)$$

est un produit scalaire $\implies$ différentiable.

$$\lambda \text{ est une constante } \implies \lambda \|Fw - v\|^2 \text{ est différentiable.}$$

$$E(w) = TV_\varepsilon(w) + \lambda \|Fw - v\|^2$$

est une somme de fonctions différentiables $\implies E(w)$ est différentiable.

Convexité de $TV_\varepsilon(w)$ et Convexité du terme $\lambda \|Fw - v\|^2$

Tout d'abord, pour montrer la convexité, on peut montrer que $\lambda \|Fw - v\|^2$ est convexe, pour le montrer nous pouvons utiliser la définition d'une fonction convexe et en utilisant l'inégalité triangulaire. Soit w_1 et $w_2 \in (\mathbb{R}^{N^2})^3$, et $\alpha \in [0, 1]$.

Nous devons montrer que :

$$\lambda \|F(\alpha w_1 + (1 - \alpha)w_2) - v\|^2 \leq \alpha \lambda \|Fw_1 - v\|^2 + (1 - \alpha) \lambda \|Fw_2 - v\|^2$$

Exprimons $F(\alpha w_1 + (1 - \alpha)w_2)$, par linéarité de F :

$$F(\alpha w_1 + (1 - \alpha)w_2) = \alpha Fw_1 + (1 - \alpha)Fw_2$$

Maintenant, considérons le côté gauche de l'inégalité :

$$\begin{aligned} \lambda \|F(\alpha w_1 + (1 - \alpha)w_2) - v\|^2 &= \lambda \|\alpha Fw_1 + (1 - \alpha)Fw_2 - v\|^2 \\ &= \lambda \|\alpha(Fw_1 - v) + (1 - \alpha)(Fw_2 - v)\|^2 \\ &\leq \lambda (\|\alpha(Fw_1 - v)\| + \|(1 - \alpha)(Fw_2 - v)\|)^2 \quad (\text{par l'inégalité triangulaire}) \\ &= \lambda (\alpha \|Fw_1 - v\| + (1 - \alpha) \|Fw_2 - v\|)^2 \\ &\leq \lambda (\alpha \|Fw_1 - v\|^2 + (1 - \alpha) \|Fw_2 - v\|^2) \quad (\text{car } \alpha, 1 - \alpha \geq 0) \\ &= \alpha \lambda \|Fw_1 - v\|^2 + (1 - \alpha) \lambda \|Fw_2 - v\|^2 \end{aligned}$$

Ainsi, nous avons montré que la fonction $\lambda \|Fw - v\|^2$ est convexe.

Maintenant nous devons montrer que TV_ε est convexe, c'est vrai car c'est une somme de fonction convexe.

Donc TV_ε est bien convexe. Alors comme la somme de fonctions convexes est convexe, on a bien E convexe.

Coercivité de $TV_\varepsilon(w)$ et Coercivité du terme $\lambda \|Fw - v\|^2$

Soit la fonctionnelle $E(w)$ définie par :

$$E(w) = TV_\varepsilon(w) + \lambda \|Fw - v\|^2.$$

Nous devons prouver que $E(w)$ est coercive, c'est-à-dire que $E(w) \rightarrow \infty$ lorsque $\|w\| \rightarrow \infty$.

Le terme $TV_\varepsilon(w)$ est la variation totale régulière et est défini par :

$$TV_\varepsilon(w) = \sum_{i,j=0}^{N-1} \sqrt{((\partial_m w^r)_{i,j}^2 + (\partial_n w^r)_{i,j}^2 + (\partial_m w^g)_{i,j}^2 + (\partial_n w^g)_{i,j}^2 + (\partial_m w^b)_{i,j}^2 + (\partial_n w^b)_{i,j}^2 + \varepsilon)}.$$

Ici, $\partial_m w^r, \partial_n w^r, \dots$ désignent les dérivées partielles de w et w^r, w^g, w^b correspondent aux composantes de couleur (rouge, vert, bleu).

Le terme $TV_\varepsilon(w)$ est une somme de normes de gradients. et for $\varepsilon > 0$, chaque carré racine is dominée by the termes of the norme L^2 du gradient, ce qui implique :

$$TV_\varepsilon(w) \geq \sum_{i,j} ((\partial_m w)_{i,j}^2 + (\partial_n w)_{i,j}^2 + \varepsilon).$$

On peut trouver des constantes C_1 et C_2 telles que :

$$C_1 \|w\|_{H^1} \leq TV_\varepsilon(w) \leq C_2 \|w\|_{H^1}.$$

Cela montre que $TV_\varepsilon(w)$ est coercif, car $\|w\|_{H^1} \rightarrow \infty$ implique que $TV_\varepsilon(w) \rightarrow \infty$.

Le terme Fw correspond à une transformation linéaire de w . Si F a un noyau trivial, c'est-à-dire si $Fw = 0$ implique $w = 0$, alors :

$$\|Fw - v\|^2 \geq \|Fw\|^2 - 2\langle Fw, v \rangle + \|v\|^2.$$

Lorsque Fw est dominé par $\|w\|^2$ (grâce à la continuité de F), la coercivité est garantie, et on peut obtenir une estimation de la forme :

$$\|Fw\|^2 \rightarrow \infty \text{ lorsque } \|w\| \rightarrow \infty.$$

Ainsi, $\|Fw - v\|^2$ est coercif en w et croît lorsque $\|w\| \rightarrow \infty$.

La fonctionnelle $E(w)$ est la somme de deux termes coercifs :

$$E(w) = TV_\varepsilon(w) + \lambda\|Fw - v\|^2.$$

Comme $TV_\varepsilon(w)$ et $\lambda\|Fw - v\|^2$ sont tous deux coercifs, la somme $E(w)$ est également coercive. Cela peut être démontré en remarquant que si chaque terme croît lorsque $\|w\| \rightarrow \infty$, alors leur somme croît également :

$$E(w) \rightarrow \infty \text{ lorsque } \|w\| \rightarrow \infty.$$

3.3 Calcul du gradient de TV_ϵ

Le démosaïage est un défi majeur en restauration d'image. Pour y remédier, on utilise la variation totale régularisée TV_ϵ appliquée à une image couleur w . Cette méthode permet de mesurer les différences entre les canaux de couleur tout en intégrant un terme de régularisation pour améliorer la stabilité du résultat. L'objectif ici est donc de déterminer le gradient de cette variation totale.

3.3.1 Éléments préliminaires de calcul

On rappelle que l'on a :

$$\nabla w_{m,n} = \begin{pmatrix} \partial_m w_{m,n} \\ \partial_n w_{m,n} \end{pmatrix} = \begin{pmatrix} w_{m+1,n} - w_{m,n} \\ w_{m,n+1} - w_{m,n} \end{pmatrix}$$

On définit les opérateurs comme suit :

$$\begin{aligned} \partial_m : \mathbb{R}^{N^2} &\longrightarrow \mathbb{R}^{N^2} \\ (w_{m,n})_{1 \leq m, n \leq N} &\longrightarrow (w_{m+1,n} - w_{m,n})_{1 \leq m, n \leq N}, \end{aligned}$$

et

$$\begin{aligned} \partial_n : \mathbb{R}^{N^2} &\longrightarrow \mathbb{R}^{N^2} \\ (w_{m,n})_{1 \leq m, n \leq N} &\longrightarrow (w_{m,n+1} - w_{m,n})_{1 \leq m, n \leq N}, \end{aligned}$$

On définit aussi la fonction φ par :

$$\begin{aligned} \varphi_\epsilon(t) &= \sqrt{t + \epsilon} \quad \forall t > 0 \\ \varphi'_\epsilon(t) &= \frac{1}{2\sqrt{t + \epsilon}} \quad \forall t > 0 \end{aligned}$$

Nous introduirons également les adjoints des opérateurs ∂_n et ∂_m . Commençons par l'adjoint de ∂_m :

$$\begin{aligned}
\langle w, \partial_m v \rangle &= \sum_{m,n=1}^N w_{m,n} (v_{m+1,n} - v_{m,n}) \\
&= \sum_{m,n=1}^N w_{m,n} v_{m+1,n} - \sum_{m,n=1}^N w_{m,n} v_{m,n} \\
&= \sum_{m,n=1}^N w_{m-1,n} v_{m,n} - \sum_{m,n=1}^N w_{m,n} v_{m,n} \\
&= \sum_{mn=1}^N (w_{m-1,n} - w_{m,n}) v_{m,n} \\
&= \langle \partial_m^* w, v \rangle
\end{aligned}$$

Alors on obtient :

$$\begin{aligned}
\partial_m^* : \mathbb{R}^{N^2} &\longrightarrow \mathbb{R}^{N^2} \\
(w_{m,n})_{1 \leq m,n \leq N} &\longrightarrow (w_{m-1,n} - w_{m,n})_{1 \leq m,n \leq N},
\end{aligned}$$

pour l'adjoint de ∂_n :

$$\begin{aligned}
\langle w, \partial_n v \rangle &= \sum_{m,n=1}^N w_{m,n} (v_{m,n+1} - v_{m,n}) \\
&= \sum_{m,n=1}^N w_{m,n} v_{m,n+1} - \sum_{m,n=1}^N w_{m,n} v_{m,n} \\
&= \sum_{m,n=1}^N w_{m,n-1} v_{m,n} - \sum_{m,n=1}^N w_{m,n} v_{m,n} \\
&= \sum_{mn=1}^N (w_{m,n-1} - w_{m,n}) v_{m,n} \\
&= \langle \partial_n^* w, v \rangle
\end{aligned}$$

On obtient donc:

$$\begin{aligned}
\partial_n^* : \mathbb{R}^{N^2} &\longrightarrow \mathbb{R}^{N^2} \\
(w_{m,n})_{1 \leq m,n \leq N} &\longrightarrow (w_{m,n-1} - w_{m,n})_{1 \leq m,n \leq N},
\end{aligned}$$

3.3.2 Calcul du gradient;

la variation totale est donnée par la formule suivante:

$$TV_\epsilon(w) = \sum_{i,j=1}^N \sqrt{(\partial_m w^r)_{i,j}^2 + (\partial_n w^r)_{i,j}^2 + (\partial_m w^g)_{i,j}^2 + (\partial_n w^g)_{i,j}^2 + (\partial_m w^b)_{i,j}^2 + (\partial_n w^b)_{i,j}^2} + \epsilon \quad (1)$$

avec $\forall \epsilon > 0$, $w \in (\mathbb{R}^3)^{N^2}$, et $\partial_m w_{i,j} = (w_{i+1,j} - w_{i,j})$ et $\partial_n w_{i,j} = (w_{i,j+1} - w_{i,j})$.

Soit $h \in (\mathbb{R}^3)^{N^2}$, pour calculer le gradient, on s'intéresse à :

$$TV_\epsilon(w + h) - TV_\epsilon(w)$$

$$TV\epsilon(w+h) - TV\epsilon(w) = \sum_{i,j=1}^N \varphi \left(|\nabla(w^r + h^r)_{i,j}|^2 + |\nabla(w^g + h^g)_{i,j}|^2 + |\nabla(w^b + h^b)_{i,j}|^2 \right) - \varphi(|\nabla w_{i,j}^r|^2 + |\nabla w_{i,j}^g|^2 + |\nabla w_{i,j}^b|^2) \quad (2)$$

avec $\varphi(t) = \sqrt{t+\epsilon}$, $\forall t > 0$.

Soit $l \in \{r, g, b\}$,

$$|\nabla(w^l + h^l)_{i,j}|^2 = |\nabla w_{i,j}^l|^2 + 2(\partial_m w_{i,j}^l \partial_m h_{i,j}^l + \partial_n w_{i,j}^l \partial_n h_{i,j}^l) + o(|\nabla h_{i,j}^l|)$$

En revenant à l'équation (2), on a :

$$\begin{aligned} TV\epsilon(w+h) - TV\epsilon(w) &= \sum_{i,j=1}^N \varphi \left(|\nabla w_{i,j}^r|^2 + 2(\partial_m w_{i,j}^r \partial_m h_{i,j}^r + \partial_n w_{i,j}^r \partial_n h_{i,j}^r) + o(|\nabla h_{i,j}^r|) \right. \\ &\quad + |\nabla w_{i,j}^g|^2 + 2(\partial_m w_{i,j}^g \partial_m h_{i,j}^g + \partial_n w_{i,j}^g \partial_n h_{i,j}^g) + o(|\nabla h_{i,j}^g|) \\ &\quad \left. + |\nabla w_{i,j}^b|^2 + 2(\partial_m w_{i,j}^b \partial_m h_{i,j}^b + \partial_n w_{i,j}^b \partial_n h_{i,j}^b) + o(|\nabla h_{i,j}^b|) \right) \\ &\quad - \varphi(|\nabla w_{i,j}^r|^2 + |\nabla w_{i,j}^g|^2 + |\nabla w_{i,j}^b|^2) \\ &= \sum_{m,n=1}^N 2\varphi'_\epsilon \left(|\nabla w_{m,n}^r|^2 + |\nabla w_{m,n}^g|^2 + |\nabla w_{m,n}^b|^2 \right) * \left(\partial_m w_{m,n}^r \partial_m h_{m,n}^r + \partial_n w_{m,n}^r \partial_n h_{m,n}^r \right. \\ &\quad + \partial_m w_{m,n}^b \partial_m h_{m,n}^b + \partial_n w_{m,n}^b \partial_n h_{m,n}^b + \partial_m w_{m,n}^g \partial_m h_{m,n}^g + \partial_n w_{m,n}^g \partial_n h_{m,n}^g \left. \right) \\ &\quad + o(|\nabla h_{m,n}^r|) + o(|\nabla h_{m,n}^g|) + o(|\nabla h_{m,n}^b|) \end{aligned}$$

On pose $X^l, Y^l \in \mathbb{R}^{N^2}$, $\forall l \in \{r, g, b\}$

$$\begin{aligned} X_{i,j}^l &= 2\varphi'(|\nabla w_{i,j}^r|^2 + |\nabla w_{i,j}^g|^2 + |\nabla w_{i,j}^b|^2) \partial_m w_{i,j}^l \\ Y_{i,j}^l &= 2\varphi'(|\nabla w_{i,j}^r|^2 + |\nabla w_{i,j}^g|^2 + |\nabla w_{i,j}^b|^2) \partial_n w_{i,j}^l \end{aligned}$$

Nous réécrivons l'équation (2) de la manière suivante :

$$\begin{aligned} TV\epsilon(w+h) - TV\epsilon(w) &= \sum_{i,j=1}^N X_{i,j}^r \partial_m h_{m,n}^r + Y_{i,j}^r \partial_n h_{m,n}^r \\ &\quad + X_{i,j}^g \partial_m h_{m,n}^g + Y_{i,j}^g \partial_n h_{m,n}^g \\ &\quad + X_{i,j}^b \partial_m h_{m,n}^b + Y_{i,j}^b \partial_n h_{m,n}^b \\ &\quad + o(|\nabla h_{m,n}^r|) + o(|\nabla h_{m,n}^g|) + o(|\nabla h_{m,n}^b|) \\ &= \langle X^r, \partial_m h^r \rangle + \langle Y^r, \partial_n h^r \rangle + \langle X^g, \partial_m h^g \rangle + \langle Y^g, \partial_n h^g \rangle + \langle X^b, \partial_m h^b \rangle + \langle Y^b, \partial_n h^b \rangle \\ &\quad + o(|\nabla h_{m,n}^r|) + o(|\nabla h_{m,n}^g|) + o(|\nabla h_{m,n}^b|) \\ &= \langle \partial_m^* X^r, h^r \rangle + \langle \partial_n^* Y^r, h^r \rangle + \langle \partial_m^* X^g, h^g \rangle + \langle \partial_n^* Y^g, h^g \rangle + \langle \partial_m^* X^b, h^b \rangle + \langle \partial_n^* Y^b, h^b \rangle \\ &\quad + o(|\nabla h_{m,n}^r|) + o(|\nabla h_{m,n}^g|) + o(|\nabla h_{m,n}^b|) \\ &= \langle \partial_m^* X^r + \partial_n^* Y^r, h^r \rangle + \langle \partial_m^* X^g + \partial_n^* Y^g, h^g \rangle + \langle \partial_m^* X^b + \partial_n^* Y^b, h^b \rangle \\ &\quad + o(|\nabla h_{m,n}^r|) + o(|\nabla h_{m,n}^g|) + o(|\nabla h_{m,n}^b|) \end{aligned}$$

Donc:

$$\nabla TV\epsilon(w) = (\partial_m^* X^r + \partial_n^* Y^r, \partial_m^* X^g + \partial_n^* Y^g, \partial_m^* X^b + \partial_n^* Y^b)$$

4 Algorithme de Restauration d'image et Résultats

Cet algorithme consistera à minimiser :

$$u^* \in \operatorname{argmin}_{w \in (\mathbb{R}^3)^{N^2}} TV_\varepsilon(w) + \lambda \|Fw - v\|^2$$

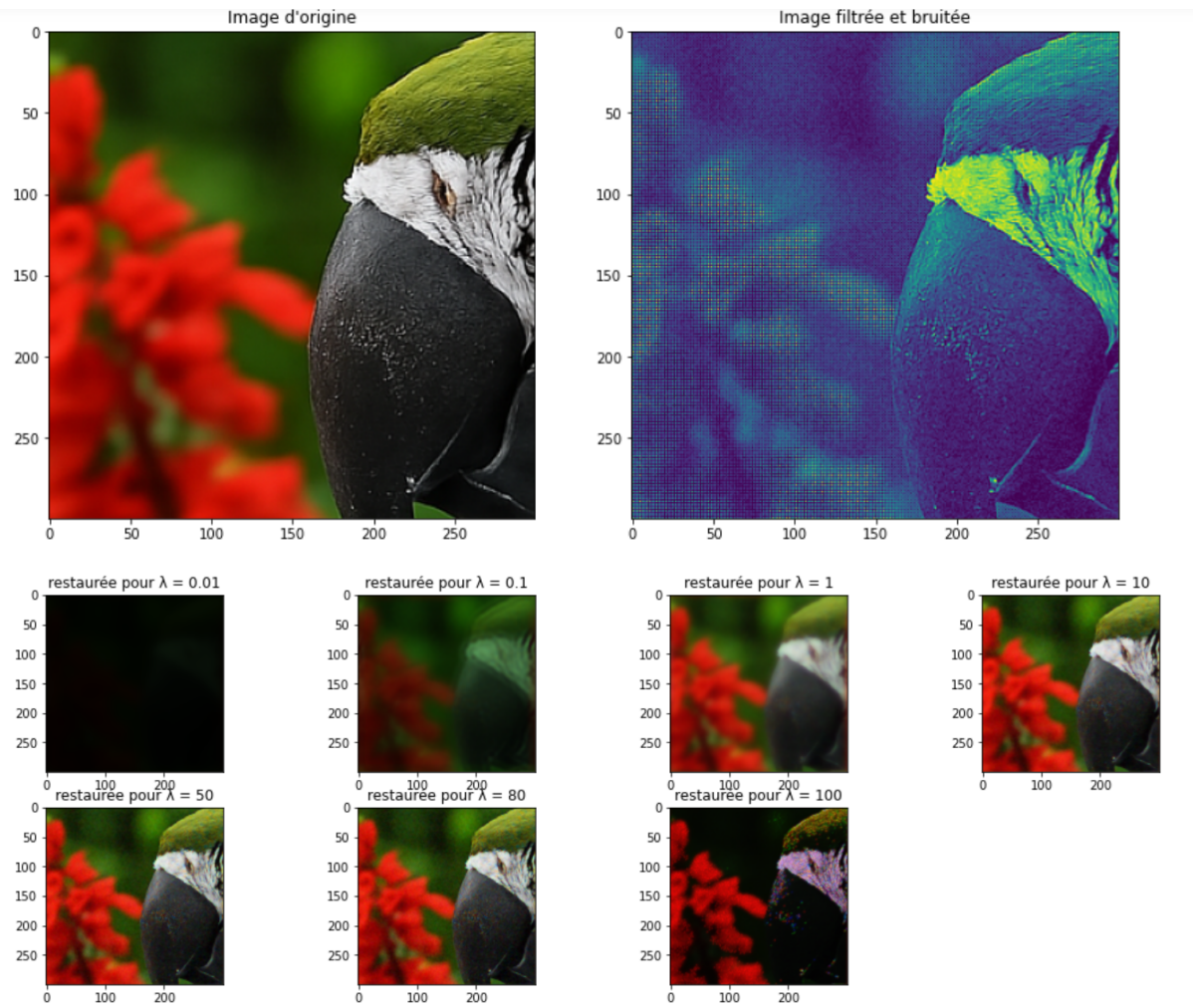
4.1 Implémentation de la fonction *démosaïcage*

Pour obtenir un résultat optimal, nous utiliserons la méthode de minimisation avec pas de Armijo. En effet cette méthode offre un compromis entre la garantie de convergence, l'efficacité des calculs et l'adaptabilité aux caractéristiques de la fonction objectif. Ainsi nous implémentons le code suivant:

```
1 def pas_armijo(img, v, E, grad_E, d, Lambda, cst=0.5, beta=0.5):
2     pas = 10
3     # Calcul nécessaire dans la condition pour ne pas le répéter chaque fois
4     Energie = E(img, v, Lambda)
5     gradienE = grad_E(img, v, Lambda)
6     k, l = img.shape[:2] # Changement de n et m en k et l
7     gradien_new = gradienE.reshape(k * l, 3) # Transformation de la matrice pour le
8     calcul de la norme
9     norme = np.linalg.norm(gradien_new, 'fro') ** 2
10
11     it = 0
12     while ((Energie - E(img + pas * d, v, Lambda)) <= pas * cst * norme) and (it < 10):
13         pas *= beta
14         it += 1
15     return pas
16
17 def demosaicage(v, img_reference, Lambda, nbIter, beta=0.5, c=0.5):
18     out = np.zeros((v.shape[0], v.shape[1], 3))
19     Energie = []
20     for it in range(nbIter):
21         d = -grad_E(out, v, Lambda) # Direction de descente (négatif du gradient)
22         step = pas_armijo(out, v, E, grad_E, d, Lambda, c, beta) # Calcul du pas selon
23         Armijo
24         out = out + step * d # Mise à jour de l'itération (ajout du gradient dans la
25         direction négative)
26
27         # Calcul de la différence par rapport à l'image de référence
28         diff = np.linalg.norm(out - img_reference)
29         Energie.append(E(out, v, Lambda))
30         #if it % 2 == 0 :
31             #print('iter =', it, 'fonction =', E(out, v, Lambda), 'distance a la
32             solution =', diff)
33     return out, Energie
```

Listing 3: fonction Demosaicage

Pour vérifier nos résultats , nous considérons l'image test :

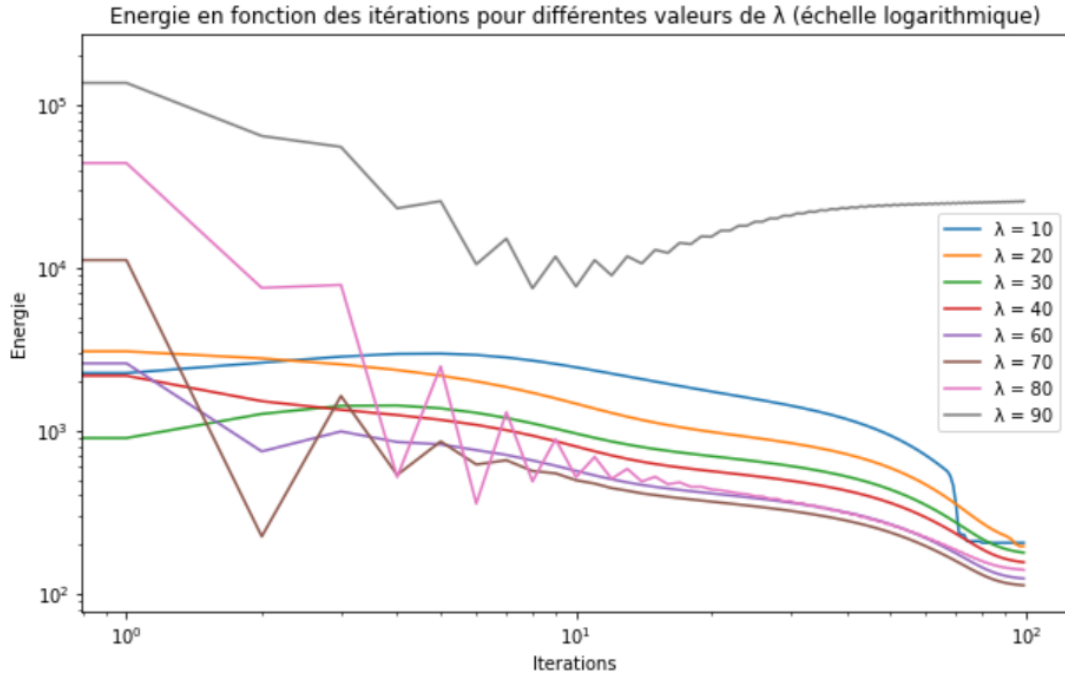


Nous faisons varier les valeurs de λ selon l'ensemble $\{0,01, 0,1, 1, 10, 50, 80, 100\}$, ce qui nous permet d'obtenir les images suivantes.

On observe que pour $\lambda \leq 1$, les images apparaissent très floues, tandis que pour $\lambda \geq 100$, elles restent insuffisamment démosaïquées . Afin d'affiner le choix du paramètre optimal λ , nous analysons alors les valeurs de E pour des λ compris entre 10 et 100.

4.2 Convergence des algorithmes

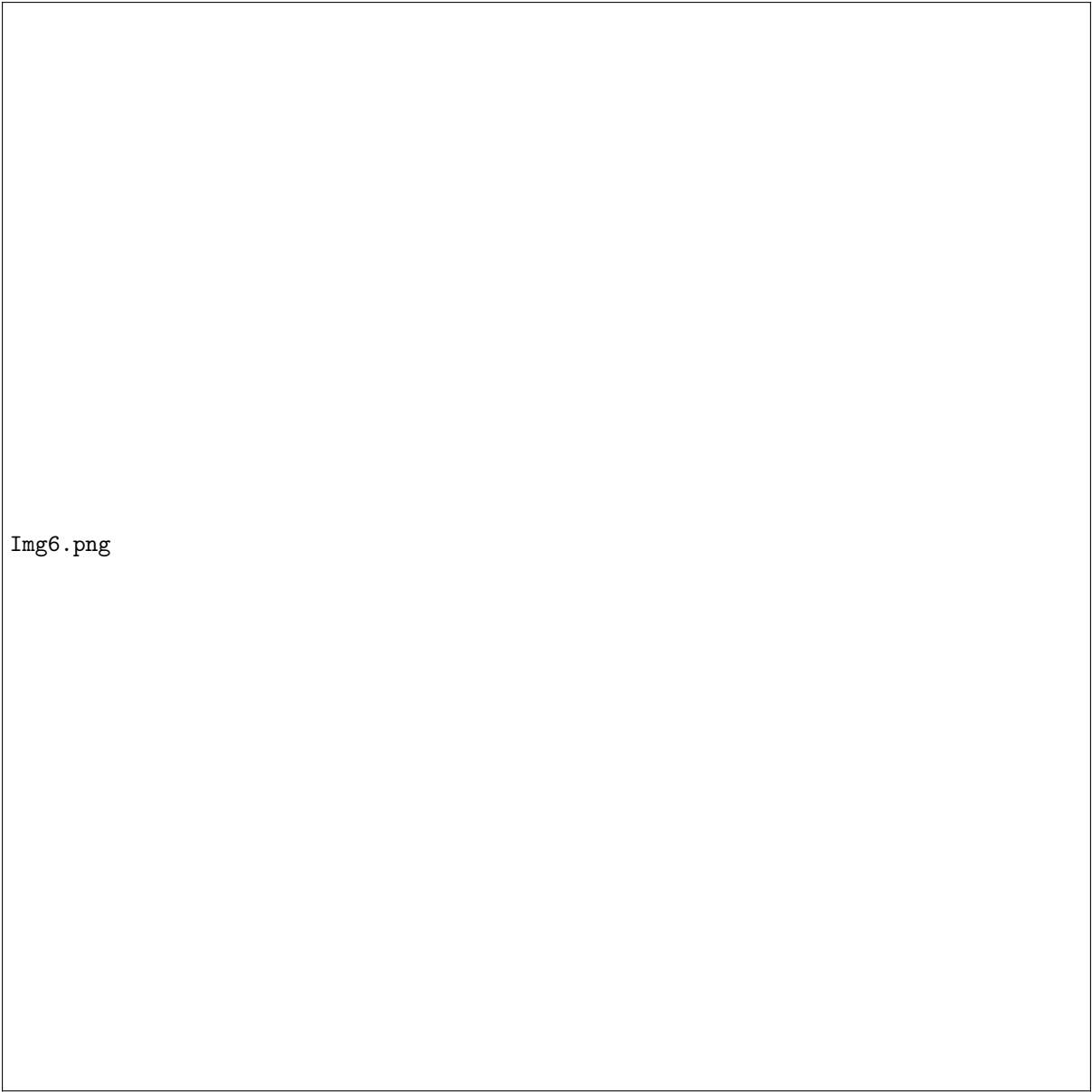
Le pas d'Armijo peut assurer la convergence de l'algorithme de descente de gradient. Toutefois, cette convergence dépend aussi d'autres éléments comme la régularité de la fonction objectif, la condition initiale et les caractéristiques du filtre de Bayer.



Ici, avec $\lambda = 80$, on obtient la meilleure minimisation de l'énergie, comme le montre la courbe correspondante sur le graphique en échelle logarithmique. Ainsi, nous retenons $\lambda = 80$ comme valeur optimale et l'appliquons à l'image entière.

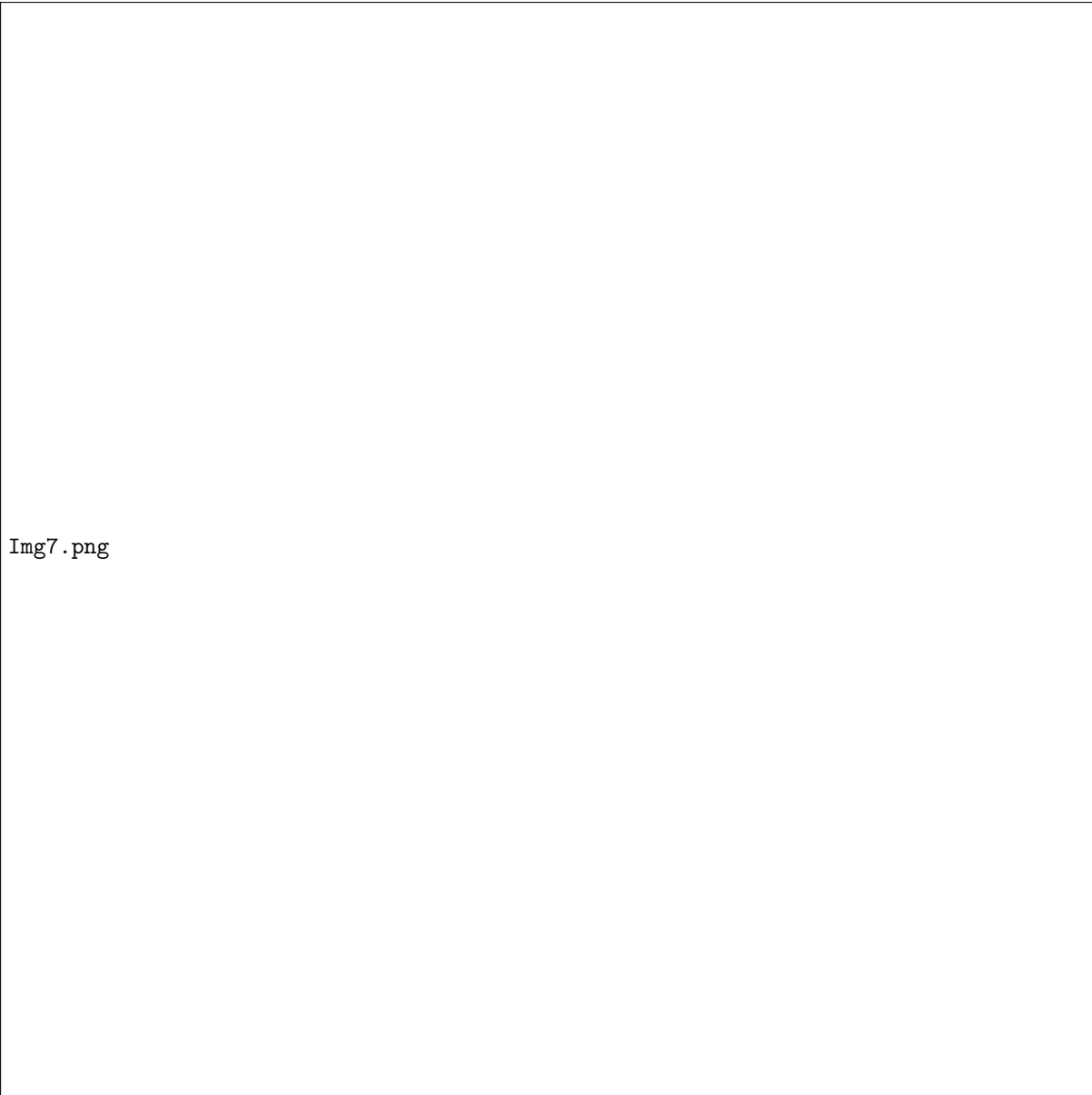
4.3 Test avec différentes Images

Nous allons tester nos algorithmes sur plusieurs images. Tout d'abord, nous afficherons l'image après application du filtre de Bayer, suivie de l'image filtrée et bruitée, puis de l'image sous forme de mosaïque. Enfin, nous procéderons à la restauration de ces images.



Img6.png

On fait de même avec une dernière image avec des couleurs moins vives



Img7.png

5 Conclusion

Pour conclure, le dématricage d'images couleurs est un domaine complexe et crucial du traitement d'images numériques. Ce processus, utilisé pour reconstruire des images en couleur à partir des échantillons de couleur incomplets, comme le filtre Bayer, nécessite l'application des méthodes mathématiques et des algorithmiques afin d'inverser le filtrage initial et restituer des images complètes. En explorant ce sujet, nous avons identifié l'importance de préserver la résolution et la qualité des images tout en minimisant les changements d'intensité non naturels, les alias chromatiques. L'algorithme de dématricage est lourd, il prend beaucoup plus de temps à l'exécution sur une image donc ça ne serait très long par exemple pour une vidéo. Ce projet a contribué à enrichir notre compréhension du démosaïquage images couleur en proposant une approche novatrice et efficace dans des diverses applications telles que l'imagerie médicale, la photographie numérique et la vision par ordinateur. Par ailleurs, ce domaine reste en constante évolution, offrant des opportunités continues d'innovation pour répondre aux besoins croissants de la technologie d'imagerie numérique.

6 Sources

– Polycopié du cours d'Image Simulation de Monsieur François Malgouyres :

https://www.math.univ-toulouse.fr/~fmalgouy/enseignement/downloadSignal_Image_Simulation/poly_image.pdf

- <https://www.photograpix.fr/blog/trucs-et-astuces/presentation-dematricage/>
- <https://en.wikipedia.org/wiki/Demosaicing>